CONCORD

*Built on the Zama FHE protocol*

# CONCORD

***Trade without showing your hand.***

`Ethereum Sepolia · euint64 sealed orders · EIP-712 user-decryption`

# THE PROBLEM WITH *open order books*

ON A TRANSPARENT DEX

## FRONT-RUN

**The MEV tax on every fill**

Your order sits in the mempool before it executes — long enough for a searcher to trade ahead of it.

01 **Visible mempool**

Every pending order is public before it settles. Searchers read it, front-run it, and sandwich it.

02 **Limit prices leak intent**

An exposed limit price is a signal. It tells the market exactly where you value the asset before you fill.

03 **Naive batching isn't enough**

Plain batch auctions fix timing, but if the book is still *readable*, the order flow is still exploitable.

# ONE SEALED BOOK, *one price*

If no one can read your order before it clears, there is nothing to front-run. The contract discovers the clearing price under FHE — without ever seeing an individual bid.

01 **Sealed submission**

Price and size are FHE ciphertexts before they touch the mempool. Only the side is public — the market sees a slot fill.

02 **Uniform price under FHE**

On close the contract computes one clearing price over the encrypted book — everyone crosses at the common price.

03 **Confidential settlement**

Crossing orders settle as ERC-7984 transfers. You decrypt only your own fill — no one reads anyone else's.

04 **32-tick price grid**

Limit prices snap to a discrete on-chain grid ($0.01 → $3.11), keeping the encrypted math inside euint64.

encrypted client-side • keeper clears the sealed book • you decrypt only your own fill

BATCH LIFECYCLE

# FIVE STATES, *keeper-driven*

A batch opens for a fixed window. A keeper drives every transition — closing the batch, scanning the encrypted grid in gas-sized chunks, public-decrypting the winner, and settling confidentially.

`Open` → `Closed` → `Clearing` → `Cleared` → `Settled`

01 **Open → Closed**

Encrypted orders accumulate for the batch window; at the deadline the keeper closes submission.

02 **Clearing**

The contract scans the 32-tick encrypted price grid in paginated, gas-sized chunks under FHE.

03 **Cleared**

The winning tick is public-decrypted — the one uniform price plus the matched volume, an aggregate only.

04 **Settled**

Crossing orders move as confidential ERC-7984 transfers. Each trader decrypts their own fill.

TECHNICAL DETAIL

# ARCHITECTURE, *the full sequence*

Every step of one batch's life, start to finish — four actors, no step skipped, the same sequence verified in `README.md`.

| | | |
|---|---|---|
| 01 | **Trader → DEX** | `submitOrder(side, encSize, encPrice, proofs)` |
| 02 | **DEX** | `FHE.fromExternal + allowThis + allow(trader)` |
| 03 | **Keeper → DEX** | `closeBatch() — window elapsed` |
| 04 | **Keeper → DEX** | `clearBatchRange(tickStart, tickEnd) · loop to MAX_TICK` |
| 05 | **DEX** | `demand/supply per tick → min() → argmaxStep()` |
| 06 | **Keeper → DEX** | `finalizeClearing()` |
| 07 | **DEX** | `makePubliclyDecryptable(bestVol, bestTick)` |
| 08 | **Keeper → Relayer** | `public-decrypt the two handles` |
| 09 | **Relayer → Keeper** | `cleartexts + KMS signatures` |
| 10 | **Keeper → DEX** | `submitClearingResult(handles, cleartexts, proof)` |
| 11 | **DEX** | `checkSignatures() → BatchCleared(tick, volume)` |
| 12 | **Keeper → DEX** | `settleBatchRange(start, end) · loop to orderCount` |
| 13 | **DEX** | `select(fill, amount, 0) confidential transfer` |
| 14 | **DEX → Trader** | `BatchSettled — filled or not, never revealed on-chain` |
| 15 | **Trader → Relayer** | `EIP-712 user-decrypt → your price · size (only yours)` |

Trader • BatchAuctionDEX • Off-chain Keeper • Zama Relayer / KMS

LIVE DEMO

# END-TO-END *on Sepolia*

Connect a wallet and run the full loop — six steps, no leaving the app.

01 **Claim from the faucet**

Mint confidential test tokens to your wallet in one click.

02 **Approve the DEX operator**

Authorize the DEX to move your ERC-7984 balances.

03 **Encrypt & submit**

Pick a side + tick + size; encrypt client-side; submit the sealed order.

04 **Batch closes**

Watch the sealed book fill — slots, never contents — then the window shuts.

05 **Keeper clears**

One uniform price is revealed over the encrypted book.

06 **Decrypt your fill**

EIP-712 user-decryption reveals only your own result.

COVERAGE



# ENTER *the app*


**Trade without showing your hand.** The landing lays out the whole thesis: sealed submission, one uniform price, confidential settlement.

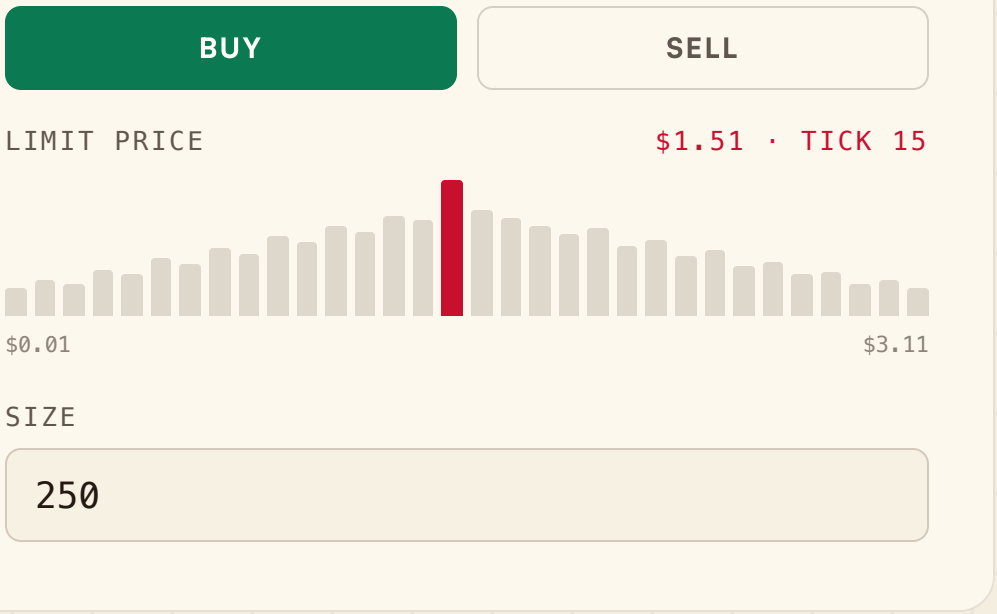
**connect → trade → view batches**

A cream "ledger-paper" interface — every number in mono, nothing about your order ever rendered to anyone else.

CORRECTNESS



# SEAL *your order*




**Pick side, tick, and size.** The limit price snaps to a discrete 32-tick grid — the same `TickMath` the contract runs.

**side • tick • size → encrypt**

The tick index (0—31) and the size are both FHE-encrypted in the browser as `euint64` ciphertexts. Only the side stays public.

CONFIDENTIALITY

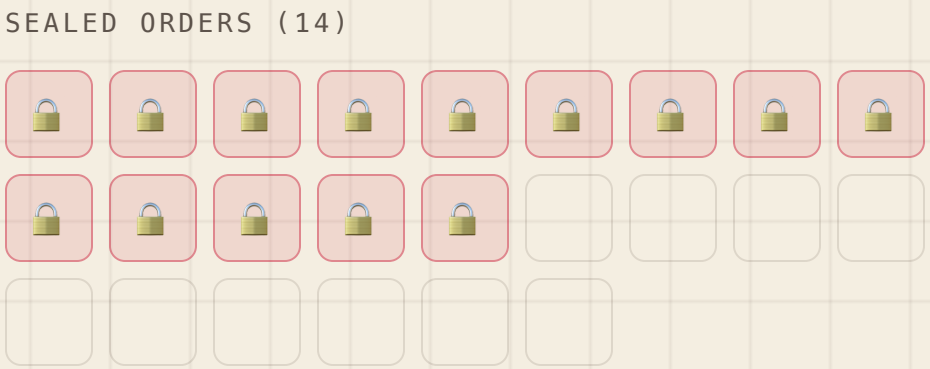


# THE SEALED *book*

**The market watches slots fill — never their contents.** Each locked cell is one order in the batch. Price and size stay encrypted on-chain.

**order in → 🔒 slot → book grows**

This is the whole point: there is no readable order flow to trade against. A searcher sees a count go up, and nothing else.



CORRECTNESS



# CLEAR *at one price*

UNIFORM CLEARING PRICE

$1.51

matched volume 1,840

**Everyone who crosses trades at the same price** — not their own bid. Under sealed submission that's incentive-compatible: you bid your true value.

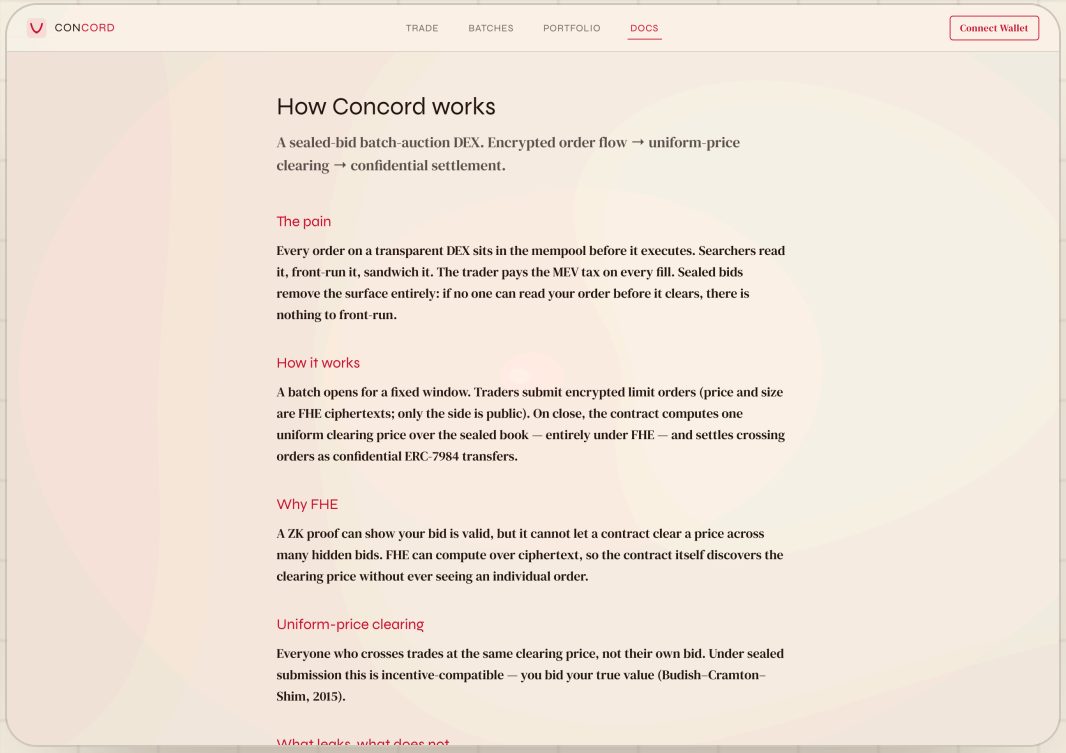
**scan grid → public-decrypt winner**

Clearing reveals only the winning tick and the matched volume — an aggregate. Individual orders never leak. ***(Budish—Cramton—Shim, 2015.)***

CORRECTNESS · EIP-712



# DECRYPT *your fill*




Your own fill is revealed through the **EIP-712 user-decryption** flow against the relayer.

**sign once → session cached → your fill**

Decrypting anyone else's fill fails the on-chain access check. The public only ever saw a ciphertext handle.

THE CORE GUARANTEE

# YOU CAN'T READ *my order*

WHAT YOU CAN DECRYPT

### Your own fill

An EIP-712 request signed by your wallet authorizes decryption of handles the on-chain ACL granted to *you*.

WHAT YOU CANNOT

### Anyone else's order

Point the same flow at another trader's handle and it fails the access check. Privacy is enforced on-chain, not by the UI.

FHE + on-chain ACL • the mempool only ever saw an encrypted handle

EXTENSIBILITY

# 32 TICKS NOW, *parameterized*

The grid is a constant, not a hard-coding. Widening it is a parameter change — the encrypted math and the keeper's chunked scan already generalize.

## A • Price grid — `lib/ticks.ts`

```
// widen the grid → more ticks
export const MAX_TICK = 31
export const TICK_SPACING_WEI = 10n ** 17n
export const MIN_PRICE_WEI = 10n ** 16n
```

## B • Indexer — metadata only

```
// never price / size — those stay
// encrypted on-chain
batch: { id, status, orderCount,
clearingTick, matchedVolume }
order: { batchId, trader, side }
```

read-only indexer • chain → REST + socket • the sealed fields are never persisted off-chain

# JUDGED *on*

### 01 Confidentiality

Price and size are euint64 ciphertexts end-to-end; only the side and post-clear aggregates are ever public.

### 02 Correctness

Encrypt → submit → clear → settle → user-decrypt produce the expected on-chain results; 11/11 contract tests pass.

### 03 MEV-resistance

No readable order flow before clearing — nothing to front-run or sandwich, by construction.

### 04 UX

Faucet, operator approval, sealed submit, live batch fill, and clearing reveal — every state animated, never a raw revert.

### 05 Code quality

Typed end-to-end; FHE isolated in hooks; keeper + read-only indexer; metadata-only off-chain store.

### 06 Production-readiness

Sepolia + Zama relayer, public-RPC fallback, keeper in docker-compose, resumable paginated clearing.

CONCORD

*Sealed bids, uniform price, confidential settlement.*

# THANK *you*

**github.com/vihaan1016/Concord**

Not affiliated with Zama. Built on the public Zama Protocol.